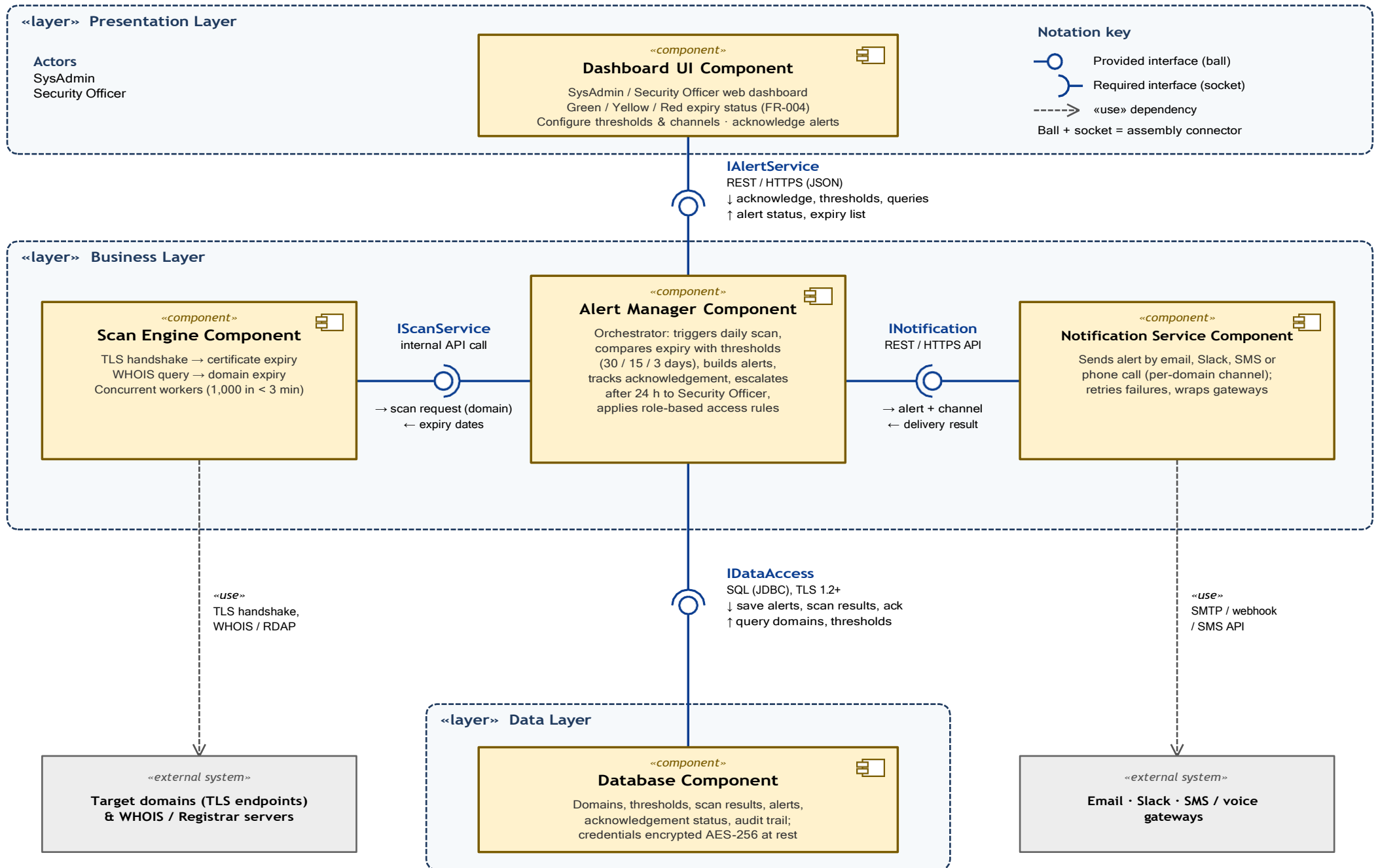


UML Component Diagram – Domain & SSL Certificate Expiry Alert System

Lab 3 | Problem #47 | Sampath | PES1UG24CS416 | Section G | Architecture: 3-tier Layered | 5 components, 4 interfaces



Each layer depends only on the layer below. All four interfaces are centred on the Alert Manager, which orchestrates the other components.

LAB 3: COMPONENT MODELLING & ARCHITECTURAL PATTERN SELECTION

Student Name: Sampath / SRN: PES1UG24CS416 / Section: G

GitHub Repository Link: <https://github.com/Sampath-404/SE-Labs-PES1UG24CS416>

Lab 3 – Supporting Analysis (Steps 1-3)

Domain & SSL Certificate Expiry Alert System (Problem #47, Developer Tools & IT Operations) | Sampath | PES1UG24CS416 | Section G | Repo: github.com/Sampath-404/SE-Labs-PES1UG24CS416 (folder Lab3)

1. Scenario review – key requirements and challenges

Functional (from Lab 1): daily TLS-certificate and WHOIS domain-expiry scans (FR-001, FR-002); alerts at 30/15/3 days; escalation to the Security Officer if a SysAdmin does not acknowledge within 24 h (FR-003); colour-coded dashboard (FR-004); per-domain thresholds and channels (FR-005). **Non-functional:** scan 1,000 domains in under 3 minutes (NFR-001); credentials encrypted with AES-256 at rest and all traffic over TLS 1.2+ (NFR-002).

Main challenges: **Performance** - many network lookups must finish inside the daily window. **Reliability** - no expiry alert or escalation may be lost. **Security** - the tool stores registrar/API credentials, so it must not become a liability.

Maintainability - WHOIS/registrar sources and notification channels (email, Slack, SMS) are external and change. **Usability** - two roles need clear, at-a-glance status.

2. Architectural style analysis

Style	Pros	Cons	Fit for this system
Layered (selected)	Clear separation of UI, rules and data; each layer testable alone; one central place for access control; simple to deploy.	Small overhead passing through layers; adjacent layers coupled; scaling is per tier.	Best fit. Workflow is a one-directional pipeline (scan → evaluate → alert → acknowledge/escalate); five components suit a small IT-ops team.
Microservices	Independent scaling and deployment, fault isolation, technology diversity.	Operational complexity (discovery, monitoring per service); network latency; distributed consistency of alert state.	Poor fit. The 24-hour acknowledgement / escalation logic needs one consistent alert state; too heavy for five components.
Client-Server	Centralised control, simple deployment, easy data consistency.	Single point of failure, scaling bottleneck, says nothing about the server's internal structure.	Partial fit. Valid as the deployment view (browser ↔ server), but the server still needs a layered structure.

3. Components (5) and interfaces (4)

Component	Layer	Responsibility	Provides / Requires
Dashboard UI	Presentation	Dashboard with Green/Yellow/Red status; threshold and channel configuration; acknowledge alerts. No direct database access.	Requires IAlertService
Alert Manager (given: Order Manager analogue)	Business	Central orchestrator: schedules daily scan, compares expiry dates with thresholds, creates alerts, tracks acknowledgement, escalates after 24 h, enforces roles.	Provides IAlertService. Requires IScanService, INotification, IDataAccess
Scan Engine (given: Payment Service analogue)	Business	Performs TLS handshakes and WHOIS lookups concurrently and returns expiry dates; hides target/registrar details.	Provides IScanService
Notification Service	Business	Delivers alerts over email, Slack, SMS or phone call; retries failed sends; wraps external gateways.	Provides INotification
Database	Data	Stores domains, thresholds, scan results, alerts, acknowledgement status, audit trail and encrypted credentials.	Provides IDataAccess

Interface	Provided (ball)	Required (socket)	Technology	Data flow
IAlertService	Alert Manager	Dashboard UI	REST / HTTPS (JSON)	UI sends acknowledgements, threshold changes, status queries; manager returns alert status and expiry list.
IScanService	Scan Engine	Alert Manager	Internal API call	Manager passes domain list; engine returns certificate and domain expiry dates.
INotification	Notification Service	Alert Manager	REST / HTTPS API	Manager sends alert, recipient and channel; service returns delivery result.
IDataAccess	Database	Alert Manager	SQL (JDBC) over TLS	Manager saves and queries domains, thresholds, scan results, alerts and acknowledgement status.

Decision: Layered Architecture is selected (Presentation → Business → Data). The Alert Manager plays the orchestrator role of the handout's Order Manager, and the Scan Engine plays the role of the specialised Payment Service.

Architectural Justification

System: Domain & SSL Certificate Expiry Alert System | Sampath | PES1UG24CS416 | Section G

Architecture Selection: *“We chose Layered Architecture for the Domain & SSL Certificate Expiry Alert System.”*

Architectural Choice

The system follows a 3-tier Layered Architecture: a **Presentation Layer** (Dashboard UI for the SysAdmin and Security Officer), a **Business Layer** (Alert Manager, Scan Engine, Notification Service) and a **Data Layer** (Database). The Presentation Layer depends only on the Business Layer, and only the Business Layer depends on the Data Layer. The Alert Manager orchestrates the other business components through four interfaces: `IAlertService`, `IScanService`, `INotification` and `IDataAccess`.

Two Reasons

1. The layers match the way an expiry alert actually flows. Every alert moves in one direction: the Scan Engine reads certificate and WHOIS expiry dates (FR-001, FR-002); the Alert Manager compares them with each domain's thresholds (FR-005) and creates the alert; the Notification Service delivers it; and the SysAdmin acknowledges it on the dashboard (FR-004) or, after 24 hours, the Alert Manager escalates it to the Security Officer (FR-003). Each step is owned by exactly one component, so a WHOIS provider or a notification channel (email, Slack, SMS) can be replaced by changing one component without touching the alert rules.

2. It is right-sized for an IT-operations tool. The scope is five components and four interfaces. Microservices would add service discovery and distributed data consistency — a real risk here, because the 24-hour acknowledgement/escalation logic depends on one consistent alert state. Client-Server alone is too coarse: it explains the browser-to-server split but not how the server itself is structured. A layered design is simple to build, test and deploy for a small team.

Security Advantage

Every dashboard request must pass through the Alert Manager's `IAlertService`, giving a single place to enforce authentication and role-based access (only a SysAdmin may change thresholds; the Security Officer receives escalations). The Database is reachable only through `IDataAccess` from the Business Layer, so stored registrar/API credentials, encrypted with AES-256 at rest (NFR-002), can never be read by bypassing business rules, and the browser never holds database credentials. All interface traffic and outbound scans/notifications use TLS 1.2 or higher (NFR-002).

Performance Benefit

Scanning is isolated behind `IScanService`, so the Scan Engine can run TLS handshakes and WHOIS lookups concurrently with a pool of workers, which is what allows 1,000 domains to be checked in under 3 minutes (NFR-001) since the work is network-wait-bound. Notification delivery sits behind `INotification` and runs asynchronously, so a slow SMS gateway never delays the scan. The Alert Manager is stateless and can be replicated, and per-domain thresholds can be cached in the Business Layer to reduce database round trips.

Trade-off Accepted

Passing requests through layers adds a small overhead and adjacent layers are coupled through their interfaces. This is acceptable because the request path is short (UI → Alert Manager → Database) and the interfaces are small and stable.